

Chapter 5: Architecture and Process Model of the RS-DRL-Based Self-Adaptive System (Alternative Structure)

Thesis Documentation

February 8, 2026

Contents

1 Chapter Overview and Architectural Perspective

1.1 Chapter Scope and Objectives

This chapter presents the **architecture and process model** of the RS-DRL-based self-adaptive system, focusing on system-level organization, runtime control flow, and component interaction patterns. The chapter emphasizes architectural responsibility and design rationale, maintaining a clear separation between system architecture (presented here) and algorithmic mechanisms (detailed in Appendices A–C).

The chapter addresses three primary objectives:

1. **Architectural organization:** Define the structural composition of the managing system and its integration with the managed system
2. **Runtime process flow:** Explain how runtime data flows through monitoring, analysis, planning, and execution phases
3. **Design rationale:** Justify architectural decisions regarding temporal separation, process boundaries, and component coordination

1.2 Process-Oriented Organization and Design Rationale

This chapter adopts a **process-oriented organization**, treating the RS-DRL adaptation process as the dominant end-to-end process that governs system behavior. The primary organizing principle is **process progression**—each section answers: “What happens next in the system, and why?” MAPE-K serves as an **explanatory lens**, not a competing organizational structure.

The chapter deliberately separates:

- **Architectural concerns** (what components exist, how they interact, when processes execute)
- **Algorithmic concerns** (how learning algorithms work, how verification is computed)

This separation reflects a fundamental thesis design principle: the architecture defines **who talks to whom, when, and why**, while the methodology defines **how optimization and learning happen**. Algorithmic and statistical details are available in Appendices A–C for readers requiring implementation-level transparency.

1.3 Architectural Baseline and Traceability Principles

The architectural baseline is the RS-DRL integration architecture (Figure 4 of the original article), which establishes the foundation for process-oriented refinement. This chapter introduces a **nine-stage adaptation flow (Stages 1–9)** that provides traceability throughout all subsequent sections.

The primary architectural baseline is shown in Figure ??, which serves as the reference for the process-oriented refinement presented throughout this chapter.

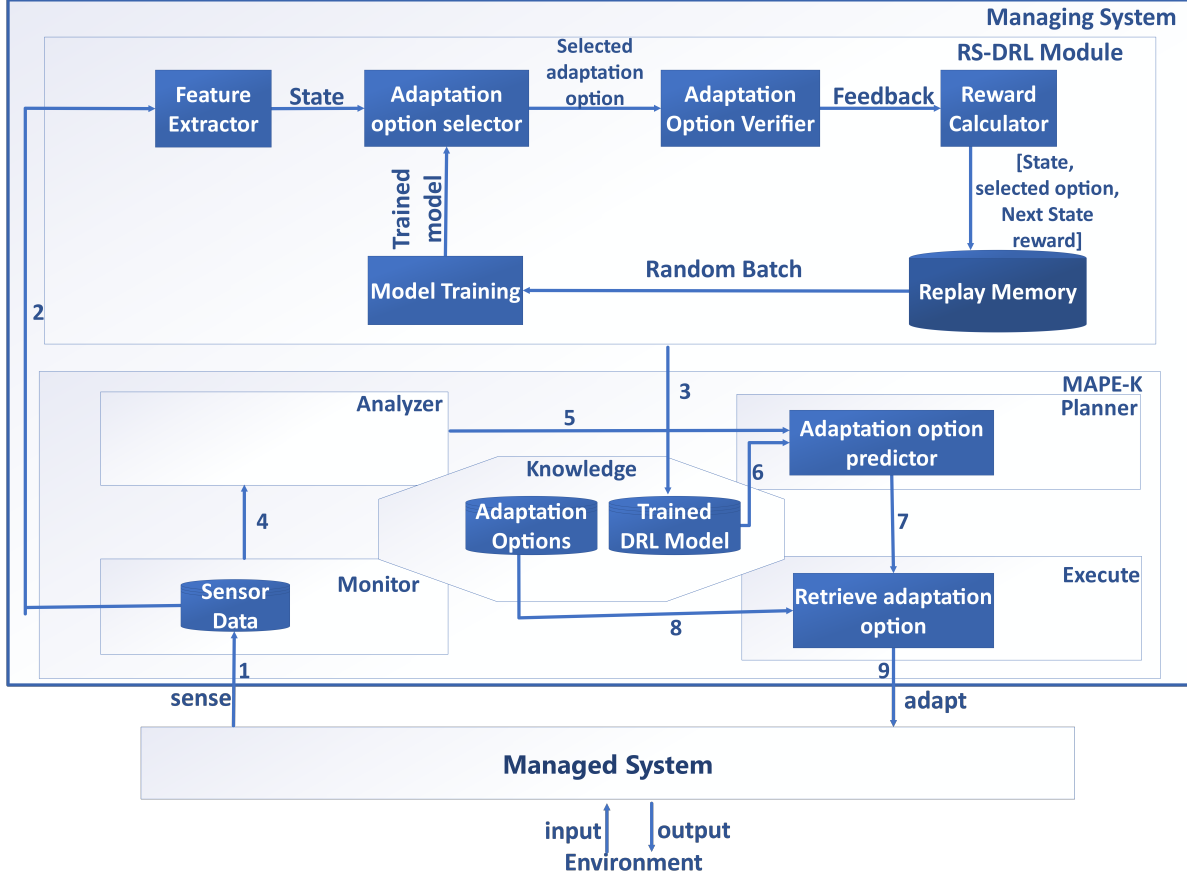


Figure 1: RS-DRL integration architecture (Figure 4 from the original article). The primary architectural baseline showing the integration of RS-DRL with the MAPE-K framework for self-adaptive systems.

2 System-Level Self-Adaptation Loop

2.1 Managed System and Managing System Abstraction

The RS-DRL-based self-adaptive system consists of two main architectural components: the **Managed System** and the **Managing System**. The Managed System executes the application logic and generates runtime data through its normal operation. The Managing System implements the adaptation logic using the MAPE-K architectural pattern, enhanced with an RS-DRL module that provides learned intelligence for adaptation decisions.

This architectural separation reflects a fundamental design principle: the managed system focuses on domain-specific functionality, while the managing system focuses on adaptation control. The interface between these components is defined by runtime data streams (flowing from managed to managing) and adaptation actions (flowing from managing to managed).

2.2 Continuous Adaptation Cycle and Stage-Based Overview

At the highest level, the system performs one dominant process: continuously adapt a managed system under uncertainty using learned adaptation policies. This process decomposes into a high-level **Sense–Plan–Adapt** loop:

1. **Sense:** Monitor the environment and system state, constructing validated state vectors
2. **Plan:** Evaluate adaptation needs and select appropriate adaptation options using learned policies
3. **Adapt:** Execute selected adaptations and update system configuration

This high-level abstraction is refined through a **nine-stage adaptation flow (Stages 1–9)** that provides detailed traceability:

- **Stage 1:** Runtime data collection from the managed system
- **Stage 2:** State forwarding to RS-DRL module (for learning/inference)
- **Stage 3:** Asynchronous model training and storage (learning path)
- **Stage 4:** State forwarding to Analyzer (for goal evaluation)
- **Stage 5:** Adaptation triggering (Analyzer notifies Planner)
- **Stage 6:** Trained model retrieval from Knowledge
- **Stage 7:** Adaptation option prediction
- **Stage 8:** Adaptation option retrieval from Knowledge
- **Stage 9:** Adaptation application to managed system

All subsequent sections refine **subsets of Stages 1–9**, progressively decomposing the system-level loop into focused subprocesses.

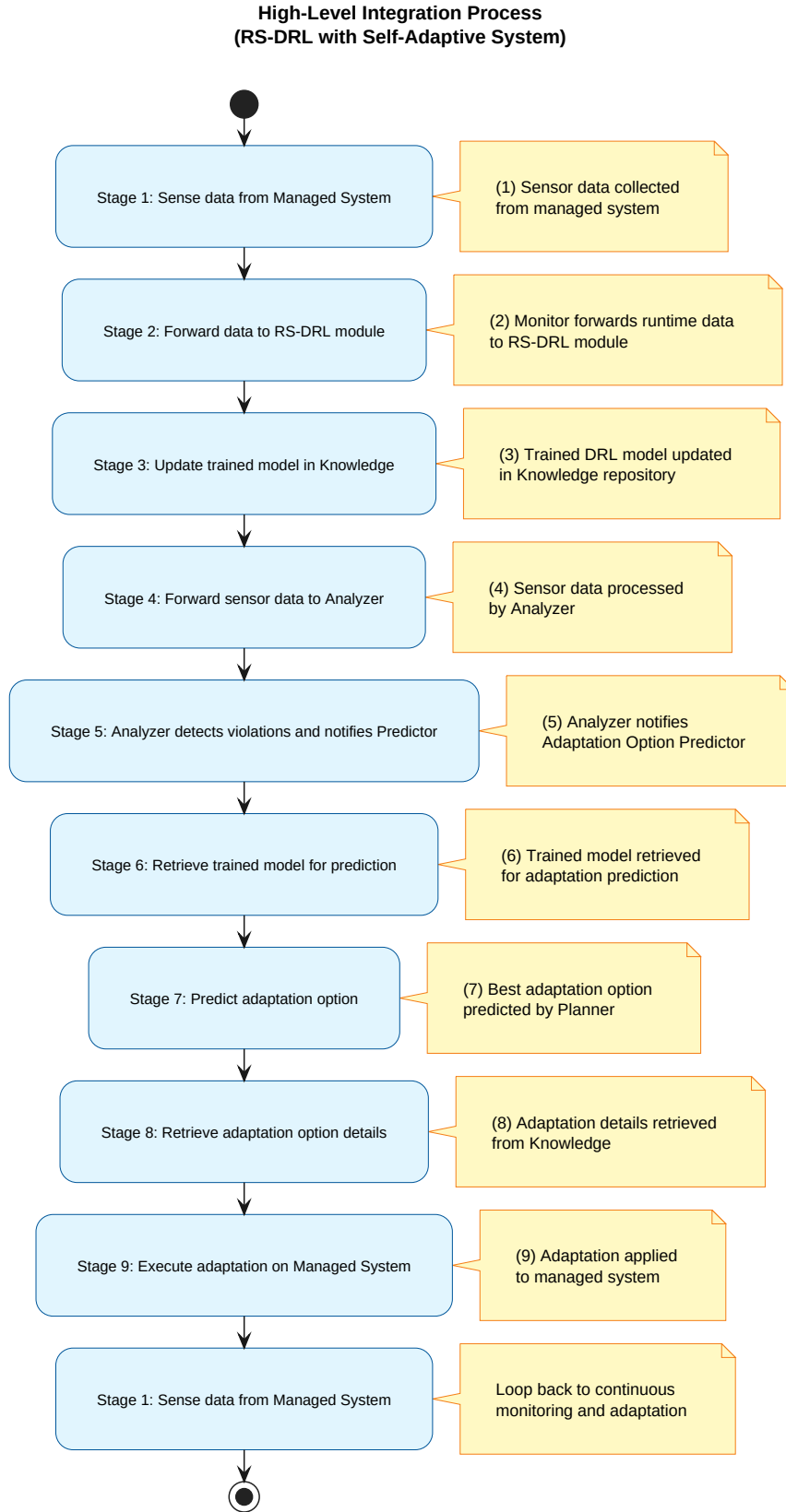


Figure 2: System-level self-adaptation process overview. This figure provides a linear, stage-based walkthrough of the RS-DRL-driven adaptation cycle, used throughout this chapter as the primary reference for process-oriented refinement.

2.3 Separation of Runtime Adaptation and Asynchronous Learning

A critical architectural design decision is the **temporal separation** between runtime adaptation (Stages 1, 4–9) and asynchronous learning (Stages 2–3). This separation enables:

- **Runtime stability:** Decision-making uses pre-trained models, ensuring predictable response times
- **Learning flexibility:** Model training occurs in the background without disrupting runtime operation
- **Scalability:** Computationally expensive training does not compete with time-critical adaptation decisions

Stage 3 (model update) occurs independently of the runtime adaptation cycle. Learning episodes are triggered by runtime state observations (Stage 2) but execute asynchronously, storing updated models in the Knowledge repository (Stage 3) for future runtime use (Stage 6).

This design reflects a deliberate architectural trade-off: we sacrifice immediate model updates for the benefit of runtime responsiveness and system stability. The temporal decoupling ensures that the system can make timely adaptation decisions even while learning processes continue to refine policies in the background.

3 Mapping RS-DRL to the MAPE-K Reference Model

3.1 Conceptual Role of RS-DRL within MAPE-K

This section provides a **conceptual mapping** of the RS-DRL architecture to the MAPE-K (Monitor, Analyze, Plan, Execute, Knowledge) architectural pattern. This mapping serves as a **classification lens**, not a process description—it categorizes the stages introduced in Section ?? according to MAPE-K functional roles.

The RS-DRL module is positioned as an enhancement of the **Plan** phase, providing learned intelligence for adaptation option selection. The module does not replace MAPE-K components but rather enriches the planning capability with data-driven decision-making.

The mapping of stages to MAPE-K components is as follows:

- **Monitor** (Stages 1, 2, 4): Observes the managed system, collects runtime data, and disseminates state information
- **Analyze** (Stage 4, 5): Evaluates state vectors against adaptation goals, detects violations, and triggers adaptation
- **Plan** (Stages 5, 6, 7): Makes adaptation decisions using RS-DRL learned policies
- **Execute** (Stages 8, 9): Retrieves adaptation specifications and applies them to the managed system
- **Knowledge** (Stages 3, 6, 8): Stores trained models, adaptation options, and state history

The RS-DRL module itself spans multiple MAPE-K phases: it receives input from Monitor (Stage 2), supports decision-making in Plan (Stages 6–7), and stores learned knowledge (Stage 3). This cross-cutting nature reflects RS-DRL’s role as an intelligent decision service rather than a standalone architectural component.

3.2 Classification of Processes by Temporal Behavior

The architectural processes can be classified by their temporal characteristics, which inform coordination and scheduling decisions:

Process	Temporal Nature
Monitoring	Continuous
Analysis	Event-driven
Prediction	Event-driven
Verification	Conditional
Execution	Event-driven
Training	Asynchronous

Table 1: Process classification by temporal nature.

This classification justifies why processes are separated: they operate with different temporal requirements and can be understood and refined independently while maintaining integration within the overall adaptation loop.

3.3 Validation View of the Adaptation Control Loop

Figure ?? provides a MAPE-K-specific validation view of the adaptation architecture, complementing the process-oriented view in Figure ?. This figure serves as a control-theoretic validation lens for examiners familiar with MAPE-K, illustrating how monitoring and analysis trigger adaptation, how planning delegates decision-making to the RS-DRL predictor, and how execution enacts selected adaptations.

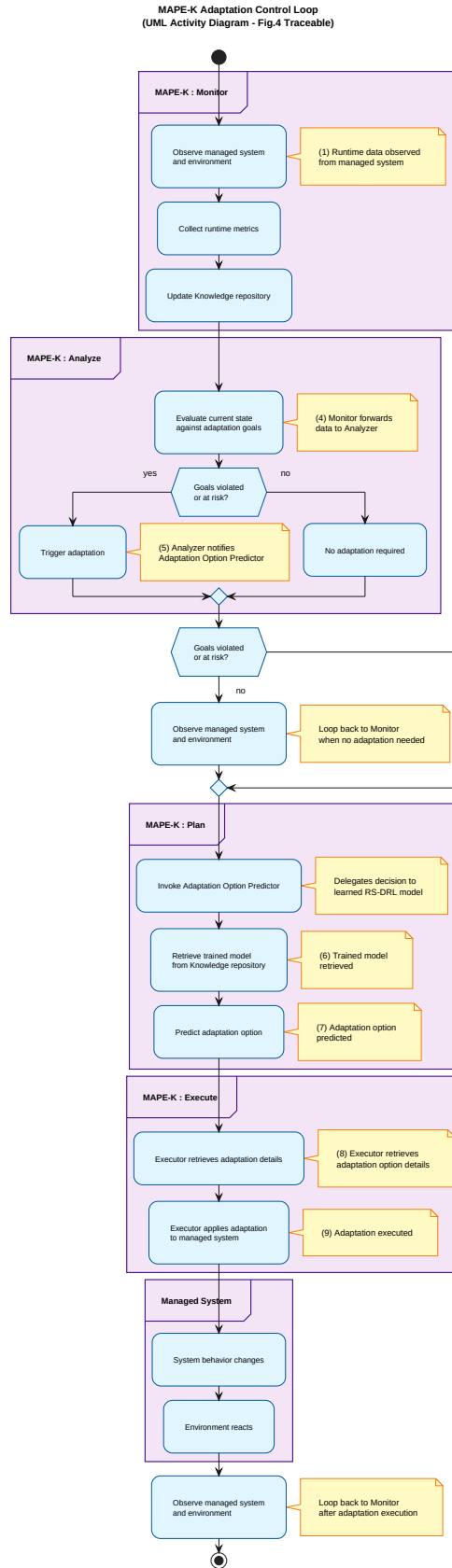


Figure 3: MAPE-K control loop summary: validation view of the adaptation architecture.

4 Monitoring and State Construction Process (Architectural View)

4.1 Runtime Sensing and State Abstraction

The monitoring process performs runtime data collection from the managed system, transforming raw sensor data into decision-ready state vectors. The process responsibility is to maintain a validated state representation that serves as the interface contract between monitoring, learning, and planning components.

The monitoring process begins with raw data acquisition (Stage 1), proceeds through preprocessing and feature extraction, and terminates with state dissemination (Stages 2 and 4). Detailed monitoring workflows, including preprocessing steps, normalization procedures, and feature extraction algorithms, are provided in Appendix A.1.

4.2 Dissemination of State Information to Analysis and Learning

Once constructed, the validated state vector is disseminated to multiple system components:

- **To Analyzer** (Stage 4): For goal evaluation and adaptation triggering
- **To RS-DRL module** (Stage 2): For learning (during training episodes) or inference (during runtime adaptation)
- **To Knowledge** (implicit): State history stored for reference

Stage 2 provides state information to the RS-DRL module for both runtime inference and asynchronous learning, without implying synchronous training during runtime operation.

4.3 Process Boundary and Separation of Concerns

Process boundary: The monitoring and state construction process ends once a validated state vector exists and has been disseminated. Everything that follows—decision-making, learning, or execution—operates on this validated state representation, maintaining clear separation between perception (monitoring) and action (decision and execution).

This boundary definition ensures that monitoring focuses solely on state observation and abstraction, delegating all decision-making responsibility to downstream processes.

5 Runtime Analysis and Adaptation Decision Flow

5.1 Goal Violation Detection and Adaptation Triggering

The **Analyzer** component coordinates goal evaluation and adaptation triggering. The Analyzer receives state vectors from the Monitor (Stage 4) and performs goal evaluation, checking whether the current system state satisfies specified adaptation goals or whether violations or risks are present.

If no violation is detected, the system continues normal operation without triggering adaptation. When the Analyzer detects that adaptation goals are violated or are no longer achievable, it notifies the Adaptation Option Predictor embedded in the Planner (Stage 5), triggering the planning phase.

The analysis process validates state vectors against adaptation goals, providing context interpretation that determines whether intervention is required. This validation ensures that adaptation decisions are made only when necessary, preserving system responsiveness during normal operation.

5.2 Delegation of Decision Making to Learned Policies

Upon receiving the adaptation trigger (Stage 5), the planning process delegates decision-making to the RS-DRL module. The Adaptation Option Predictor retrieves the trained RS-DRL model from the Knowledge repository (Stage 6) and evaluates Q-values for all available adaptation options.

The selection process is **pure inference**—it uses a pre-trained model to evaluate options but performs no learning, no replay memory sampling, and no policy updates at runtime. At runtime, the selection strategy is typically purely exploitative ($\epsilon \approx 0$), choosing the option with the highest Q-value to leverage learned knowledge for optimal decision-making.

The predicted adaptation option (Stage 7) completes the planning phase. Detailed decision workflows, including Q-value evaluation and selection mechanics, are provided in Appendix A.2.

5.3 Decision Justification Under Uncertainty (Overview)

The predicted adaptation option undergoes verification to provide decision justification under uncertainty. Verification belongs to **runtime justification**—its primary purpose is to evaluate predicted options and provide quality estimates that justify adaptation decisions.

The verification process uses runtime models and statistical model checking (UPPAAL-SMC) to estimate goal satisfaction probabilities under uncertainty. While verification also generates reward signals consumed by the learning process, reward generation is a **by-product** of verification, not its primary goal during runtime decision-making.

Detailed verification workflows, including runtime model parameterization, statistical simulation, and quality evaluation, are provided in Appendix B.

6 RS-DRL Module: Architectural Role and Interfaces

6.1 Positioning of RS-DRL within the Managing System

The RS-DRL module serves as the **innovation center** of the self-adaptive system, but it is architecturally positioned as an **internal decision service within the Planner**, not as the system architecture itself. While MAPE-K provides the architectural framework for self-adaptation, the RS-DRL module is what makes adaptation scalable, robust, and uncertainty-aware.

The module corresponds to the **RS-DRL component** shown in Figure ?? (Figure 4 of the original article). It is realized through processes described in earlier sections: runtime inference (Sections ?? and ??), with learning mechanisms detailed in Appendix C.

6.2 Runtime Inference Path and Planner Integration

The runtime inference path enables the RS-DRL module to support planning decisions without introducing computational overhead during time-critical adaptation. The path consists of:

1. **Model retrieval:** Trained model accessed from Knowledge (Stage 6)
2. **Q-value evaluation:** All adaptation options evaluated using the retrieved model
3. **Option selection:** Best option selected based on Q-values (Stage 7)

This inference path is temporally decoupled from learning: the model used for inference was trained asynchronously (Stage 3) during previous episodes and stored in Knowledge for runtime access.

6.3 Interaction with Verification and Knowledge Components

The RS-DRL module interacts with two critical architectural components:

- **Verification component:** Provides quality estimates for predicted options, enabling decision justification (detailed in Appendix B)
- **Knowledge repository:** Serves as persistent storage for trained models (Stage 3 writes, Stage 6 reads)

These interactions maintain clear architectural boundaries: the RS-DRL module focuses on decision intelligence, delegating quality assessment to verification and persistence to Knowledge.

7 Execution and System Reconfiguration Process

7.1 Retrieval of Adaptation Specifications

The **Executor** component coordinates adaptation enactment. Upon completion of the planning phase (Stage 7), the Executor retrieves the selected adaptation option from the Knowledge repository (Stage 8). The retrieval process accesses detailed adaptation specifications that define how to enact the selected option.

7.2 Adaptation Enactment and System Reconfiguration

The Executor translates the selected adaptation option into concrete system modifications (Stage 9). The enactment process applies the adaptation to the managed system, altering system configuration, resource allocation, or operational parameters according to the adaptation specification.

The Executor is responsible for ensuring that adaptations are applied correctly and safely, handling potential failures or inconsistencies that may arise during reconfiguration.

7.3 Feedback Loop Closure and Continuous Operation

After adaptation execution, monitoring resumes, and the control loop continues. The system enters a new operational state, which will be observed by the Monitor in subsequent cycles (returning to Stage 1).

This feedback loop closure ensures continuous adaptation: the system perpetually monitors, evaluates, plans, and executes, forming a closed-loop control system that continuously seeks to maintain adaptation goals under changing conditions.

8 Knowledge Management and Process Coordination

8.1 Knowledge as Shared System Memory

The **Knowledge repository** serves as shared system memory, storing trained models, adaptation options, and state history. Knowledge management is explained from a **coordination semantics** perspective, focusing on **who reads what and when**, rather than storage mechanics.

8.2 Read–Write Coordination Contracts

Knowledge coordination occurs through well-defined read-write contracts:

- **When monitoring completes** (Stage 1): State vectors stored for historical reference
- **When learning completes** (Stage 3): Trained models written, making learned policies available for future runtime use
- **When planning begins** (Stage 6): Trained models read, enabling the Adaptation Option Predictor to leverage learned knowledge
- **When execution begins** (Stage 8): Adaptation option details read, providing the Executor with specifications

8.3 Synchronization of Heterogeneous Temporal Processes

The Knowledge repository enables independent processes with different temporal natures (continuous monitoring, event-driven prediction, asynchronous learning) to synchronize effectively without tight coupling. This coordination structure ensures that all components operate on consistent information while maintaining clear separation of concerns.

9 Summary of Architectural Processes

9.1 Hierarchical Process Decomposition Recap

This chapter presented a structured process model of the RS-DRL-based self-adaptive system, organized through progressive refinement from a system-level adaptation loop (Section ??) to focused architectural views (Sections ??–??).

The RS-DRL adaptation process was defined as the dominant end-to-end process, decomposing hierarchically into:

1. System-level self-adaptation loop (black-box view)
2. MAPE-K conceptual mapping (classification lens)

3. Runtime monitoring and state construction (perception)
4. Analysis and decision flow (control)
5. RS-DRL module architecture (decision intelligence)
6. Execution and reconfiguration (action)
7. Knowledge management (coordination)

9.2 Architectural Contributions of RS-DRL-Based Adaptation

The architectural contributions of the RS-DRL-based approach include:

- **Temporal separation:** Clear decoupling of runtime decision-making from asynchronous learning
- **Process-based traceability:** Nine-stage flow enabling systematic reasoning about system behavior
- **Verification as justification:** Semantic grounding for learning-based decisions through runtime model checking
- **Knowledge as coordination:** Enabling heterogeneous processes to synchronize without tight coupling

9.3 Transition to Evaluation and Experimentation

This chapter established the architectural foundation for the RS-DRL-based self-adaptive system. The detailed algorithmic mechanisms supporting this architecture (reward-shaping, replay memory, verification workflows) are provided in Appendices A–C for readers requiring implementation-level transparency.

The next chapter (Chapter 6) evaluates this architecture through systematic experimentation, demonstrating how the architectural design decisions enable effective adaptation under uncertainty.